MCH-02: Server Nodes and Processes

> **Series:** MCH — Managed Cluster Health | **Notebook:** 2 of 8 | **Created:** September 2026 | **Last Updated:** 10/02/2026

Overview

Layer 1 of the MCH-01 health model asks the most basic question: **is every node up, and is every service on each node running?** Every other layer depends on the answer. A stopped Cassandra shows up as a storage problem, and a blocked port shows up as a connectivity problem, but both are node problems first.

This notebook covers how to see node and process state (in the Cluster Management Console, through the Cluster API, and on the node itself), how the services depend on each other, how to stop and restart them without losing redundancy, which node-level events reach you and which don't, and what to check when a node looks unhealthy.

What you'll learn:

- The two authoritative views of node state, and the one field the API documents
- The read-only commands that check every service and firewall rule on a node
- Why services must start in a fixed order, and the health gates between steps
- Why `dynatrace.sh` is the wrong tool to stop a node in a 3+ node cluster
- Which node and process events never email you
- The memory, time and network conditions that quietly take a node out

---

Table of Contents

1. Short Answer
2. Node Inventory and Operation State
3. Process Checks on a Node
4. Service Dependencies and Start Order
5. Stopping and Restarting Safely

6. Node and Process Events — What Emails You
7. Memory and Time
8. Inter-Node Network
9. When a Node Looks Unhealthy
10. What the Documentation Does Not Say
11. Recommended Approach
12. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace deployment	A self-hosted **Dynatrace Managed** cluster. Nothing in this series applies to Dynatrace SaaS.
Access	Cluster Management Console (CMC) administrator access; root (or `sudo`) on the cluster nodes; for the Cluster API, a cluster API token with the `ServiceProviderAPI` permission
Query language	None. Managed has no Grail, so this series has **no DQL cells**. Every check is a CMC view, a cluster event, a node command, or a Cluster API call
Read first	MCH-01 — the node components and the five-layer health model
Version basis	Written against the Managed documentation read 09/28/2026

1. Short Answer

- **Two views of node state.** CMC **Deployment status** is for people. `GET /api/v1.0/onpremise/cluster` is for automation, and its `operationState` field is the only node state the API documents. `RUNNING` is the only value the docs show; a Dynatrace product manager has posted the full list of 12 (§2).
- **`dynatrace.sh status` and `check` are your on-node health check.** Between them they cover the seven Dynatrace services, the firewall rules, and the listening ports.
- **Services start in a fixed order**: firewall → Elasticsearch → Cassandra → Server → everything else. Each storage layer must be healthy on every node before the next one starts.

- **At three or more nodes, don't stop a node with `dynatrace.sh`.** The docs reserve it for clusters of fewer than three nodes. Larger clusters use the cluster procedure.
- **Planned work goes one node at a time.** That applies to OS patches, updates, removals and IP changes. Three copies of the data survive one missing node, not two — a Dynatrace blog allows two only from five nodes up.
- **Most node and process events never email you.** A component going down, memory emergency mode, clock drift and a node that can't receive agent traffic all appear only in the CMC **Events** list and in Mission Control.

> **Sources:** Get cluster information about known cluster nodes (DT docs), Start/stop/restart a node (DT docs), Start/stop/restart a cluster (DT docs), Configure Cluster event notifications (DT docs).

2. Node Inventory and Operation State

2.1 In the Cluster Management Console

"Deployment status provides an overview of your Cluster infrastructure." Expand a node's row for its details. Dead nodes don't vanish at once: *"Dynatrace Managed shows dead and removed nodes for 7 days."* A node that dropped out overnight is therefore still visible in the morning.

2.2 Through the Cluster API

All four calls below are v1 and need the `ServiceProviderAPI` permission. MCH-01 §6.4 shows the request format.

Call	Returns	Fields that matter for health
`GET /api/v1.0/onpremise/cluster`	*"cluster information about known nodes"*	`id`, `operationState`, `buildVersion`, `jvmInfo`, `osInfo`
`GET /api/v1.0/onpremise/cluster/configuration`	*"the cluster nodes configuration"*	`id`, `ipAddress`, `webUI`, `agent`, `datacenter`

Call	Returns	Fields that matter for health
`GET /api/v1.0/onpremise/cluster/configuration/status`	*"the current configuration status for cluster nodes"*	`state`, with per-step states for `DOMAIN_UPDATE`, `OPERATION_STATE`, `AGENT_TRAFFIC`, `WEB_UI`
`GET /api/v1.0/cluster/maintenance`	*"details about the current cluster maintenance state"*	`reason` — the only documented value is `IP_MIGRATION`

The full list of states. The docs show only `RUNNING`. In 2020 a Dynatrace product manager posted the whole list on the Dynatrace community (state of nodes (Dynatrace community)). It is a community source, not documentation, so treat it as community practice and confirm against your own cluster:

`operationState`	Meaning, as posted
`OFFLINE`	Initial — server not running
`STARTUP`	Startup in progress
`STARTUP_CANCELED`	Startup canceled due to a severe problem
`RUNNING`	Fully operational, accepts all data, fully visible
`RUNNING_FORSAKEN`	Fully operational but invisible to all agents
`SHUTDOWN_PHASED_OUT`	Before shutdown — still processing, no longer visible to agents
`SHUTDOWN`	Shutdown in progress
`EMERGENCY`	An emergency situation, for example low memory
`STARTUP_SUSPENDED`	Basic startup finished, but not connected to the cluster or database
`DATABASE_DISCONNECTED`	Database disconnected during runtime
`UNDEFINED`	Not known
`SHUTDOWN_IMMINENT`	The state before `SHUTDOWN_PHASED_OUT`

Two practical readings: `RUNNING_FORSAKEN` is a node that is up but receiving no agent traffic, and in community practice `EMERGENCY` is read alongside the heap-memory emergency events (§7.1) — the post describes it only as *an emergency situation, e.g. low memory*. The same answer names `/rest/state` (no authentication) as a lightweight status check for synthetic monitoring.

Three uses for these calls — this series' checks built on documented fields, not documented alert rules:

1. **Node down.** Alert when any node's `operationState` is not `RUNNING`, *and* when the number of nodes returned is lower than your cluster size.
2. **Version drift.** Every node should report the same `buildVersion`. A node left on an older build after an update is a node to look at.
3. **Traffic flags.** The `configuration` call shows, per node, whether it serves the web UI (`webUI`) and accepts OneAgent traffic (`agent`). A node that looks idle may simply have had agent traffic disabled (§5.3). Check this before diagnosing load.

> **Sources:**
>
> - Cluster Management Console (DT docs), Remove a cluster node (DT docs)
> - Get cluster information about known cluster nodes (DT docs), Get cluster nodes configuration (DT docs)
> - Get cluster nodes configuration current status (DT docs), Get cluster maintenance (DT docs)

3. Process Checks on a Node

Every node carries the control script `dynatrace.sh`: *"By default, the script is located at /opt/dynatrace-managed/launcher/ ."* It takes six actions, *"(start , stop , restart , status , check , pid)"*. Three of them only report and change nothing:

Action	What it does
`status`	*"Displays a list of Dynatrace required services and the status of each, including detailed information about each service"*
`check`	*"Checks the status of iptable rules and the processes for Nodekeeper, Cassandra, Elasticsearch, ActiveGate, Watcher, and NGINX."*
`pid`	*"Displays the process ID for all Dynatrace required services that were started with the dynatrace.sh script."*

```
sudo /opt/dynatrace-managed/launcher/dynatrace.sh status
sudo /opt/dynatrace-managed/launcher/dynatrace.sh check
```

A healthy node ends `status` with *"All services are OK"*, and ends `check` with *"All rules are active."* and *"All processes are OK"*. `status` walks these systemd units, in the order the docs print them:

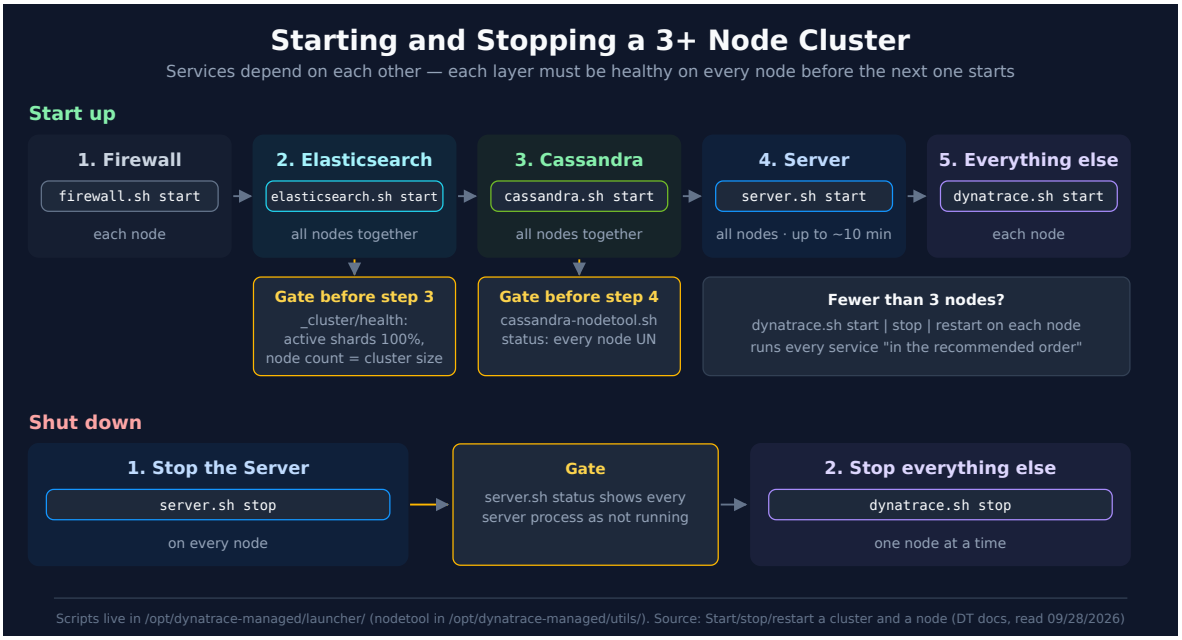
systemd unit	Description (as printed)
`dynatrace-firewall.service`	Dynatrace Firewall settings
`dynatrace-nodekeeper.service`	Dynatrace Nodekeeper
`dynatrace-cassandra.service`	Dynatrace Cassandra
`dynatrace-elasticsearch.service`	Dynatrace Elasticsearch
`dynatrace-server.service`	Dynatrace Server
`dynatrace-security-gateway.service`	Dynatrace Active Gate
`dynatrace-nginx.service`	Dynatrace NGINX

The `check` example also lists the local port each process should hold: Nodekeeper `8018`, Cassandra `9042`, Elasticsearch `9200` / `9300`, Server `8021`, ActiveGate `8443`, NGINX `8022`. In community practice, a process that is running but not listening points to a port conflict or a firewall rule rather than a crash — confirm in the node's logs.

> **Read the example output with care.** The node-script page was last updated in May 2022. Its sample output shows log paths under `/var/opt/managed/`, while current installs use `/var/opt/dynatrace-managed/`. Trust the commands, not the sample paths.
>
> **Sources:** Start/stop/restart a node (DT docs), Customize Managed Cluster installation (DT docs) — default data root `/var/opt/dynatrace-managed`.

4. Service Dependencies and Start Order

"Dynatrace Managed software consists of a number of Dynatrace services that are dependent on each other and should be stopped or started in a particular order." The Server needs both data stores, and each data store needs a quorum of its peers. So the order is fixed, and it has checkpoints.



Start-up, for clusters of three or more nodes. Run steps 2 to 4 on every node *"as simultaneously as possible"*:

1. **Firewall**, on each node: `sudo /opt/dynatrace-managed/launcher/firewall.sh start`
2. **Elasticsearch**, on every node together: `sudo /opt/dynatrace-managed/launcher/elasticsearch.sh start`. Then check it: `bash curl -X GET "localhost:9200/_cluster/health?wait_for_status=yellow&timeout=50s&pretty"`
3. **Cassandra**, on every node together: `sudo /opt/dynatrace-managed/launcher/cassandra.sh start`. Then check it: `bash sudo /opt/dynatrace-managed/utils/cassandra-nodetool.sh status`
4. **Server**, on every node together: `sudo /opt/dynatrace-managed/launcher/server.sh start`
5. **Everything else**, on each node once all server processes are up: `sudo /opt/dynatrace-managed/launcher/dynatrace.sh start`

The two gates are what make the order safe:

- **After Elasticsearch:** *"Make sure active_shards_percent_as_number is 100% and number_of_nodes is equal to the number of nodes in the cluster."*
- **After Cassandra:** *"Make sure that all nodes display UN before proceeding to the next step."*

Then allow for the Server's start time: *"It can take up to ~10 minutes to fully start a node."*

Shut-down, for clusters of three or more nodes. Stop the Server everywhere first, and stop the rest only *"When all server processes are fully shut down and a status check shows them as not running"*. Do that *"on each existing node, one node at a time"*:

1. **Server**, on every node: `sudo /opt/dynatrace-managed/launcher/server.sh stop`, then confirm with `sudo /opt/dynatrace-managed/launcher/server.sh status`
2. **Everything else**, one node at a time, once no server process is running: `sudo /opt/dynatrace-managed/launcher/dynatrace.sh stop`

> **About the paths above.** The docs print these commands as `./opt/dynatrace-managed/...`, a relative path that only works from `/`, and in one place as `../launcher/dynatrace.sh`. The commands above use the absolute path. The same scripts are called in the same order.
>
> **Sources:** Start/stop/restart a cluster (DT docs), Start/stop/restart a node (DT docs). **Derived:** the absolute-path form normalizes the docs' relative paths; the scripts and their order are unchanged.

5. Stopping and Restarting Safely

5.1 Pick the procedure by cluster size

Cluster size	Procedure
Fewer than 3 nodes	`dynatrace.sh start` / `stop` / `restart` on each node. It *"Starts all Dynatrace Managed required services in the recommended order."*
3 or more nodes	The cluster procedure in §4: *"For Dynatrace Managed deployments containing three (3) or more nodes, use the cluster procedure"*

A recorded conflict. On the Dynatrace community, a Dynatrace product manager confirmed restarting a three-node cluster one node at a time with `dynatrace.sh restart`, and described a per-node **Restart** button in the CMC (Deployment status → Cluster nodes → a node). He added that OneAgent traffic needn't be disabled for the restart (How to restart Dynatrace nodes on the cluster (Dynatrace community), 2020). The docs page says to use the cluster procedure at three or more nodes. This notebook keeps the documented procedure as the default; the community answer describes how a single-node restart is handled in practice — verify with Dynatrace support before relying on it.

The docs also say that below three nodes the cluster is exposed during maintenance: *"If your cluster consists of less than 3 nodes, Dynatrace Managed won't be accessible during the update process."*

5.2 Planned work on one node: one at a time

Every documented node operation carries the same rule:

Operation	What the docs say
OS patches	*"We recommend that you perform the node operating system update or patch application one node at a time."* After a reboot, *"Dynatrace services will automatically resume unless explicitly stopped."*
Manual update	*"Since the node that's being updated isn't operating, it's crucial that you manually update one node at a time."*
Removing a node	*"Remove no more than one node at a time. To avoid data loss, allow 24 hours before removing any subsequent nodes."*
Changing a node's IP	*"your cluster must have at least three nodes"*, and *"No, you can change only one node at a time."*
Adding a node	*"You can't add additional cluster nodes until synchronization completes."* `cassandra-nodetool.sh status` shows a joining node as `UJ`

The reason is the copy count from MCH-01: *"Dynatrace Managed continues to operate after the loss of one node."* A second node out, whether planned or not, leaves the cluster below what it was built to tolerate. Before taking a node down, confirm that every other node is healthy (§2, §3).

5.3 Draining a node without stopping it

To stop a node processing monitoring data without removing it, use the first step of the removal flow: *"Select Disable OneAgent traffic to stop monitoring data processing on the node. If you want to temporarily exclude the node from a cluster, stop here and enable the node later."* Disabling OneAgent traffic sets the node to *"idle mode"*, and the docs give two ways to do it: *"To configure OneAgent or web UI traffic on a node, use the CMC or Cluster REST API."* The API route is `POST /api/v1.0/onpremise/cluster/configuration`, which *"configures cluster nodes responsibilities"* through each node's `agent` and `webUI` flags. While agent traffic is off, the node's `agent` flag in `GET /onpremise/cluster/configuration` should therefore read `false`.

> **Sources:**
>
> - Start/stop/restart a node (DT docs), Start/stop/restart a cluster (DT docs)
> - Apply OS patches to a node (DT docs), Update a cluster (DT docs), Remove a cluster node (DT docs)
> - Reconfigure a node's IP address (DT docs), Add a cluster node (DT docs), Single-cluster high availability

> (DT docs)
>
> - Post cluster nodes responsibilities (DT docs), Configure Cluster capabilities (DT docs)
> - **Derived:** `agent: false` while traffic is off combines the capabilities page's two routes to one setting (CMC *Disable OneAgent traffic*, or the `agent` flag on `POST`) with the same `agent` field in the `GET` response

6. Node and Process Events — What Emails You

The event-notifications page gives each event an **Email notification** column (Yes / No / Configurable) and an **MC notification** column. Every row below goes to Mission Control. Only some of them email you:

Event (as the docs print it)	Severity	Emailed
"Host is down."	WARNING / SEVERE	Yes
"Node is down - %s."	WARNING	Yes, *"if happened outside of the upgrade procedure"*
"Node is up - %s." / *"Host is up."*	INFO	Yes
"Storage volume for Dynatrace Managed log files is running out of space on {0}."	SEVERE	Yes
"Your Dynatrace Managed cluster node {0} is undersized!"	SEVERE	Configurable
" is down."	SEVERE	**No**
"A cluster node can't receive OneAgent traffic."	SEVERE	**No**
"Heap memory: Server %d started memory emergency mode."	SEVERE	**No**
"Heap memory: Server %d triggered a hard memory cleanup action."	SEVERE	**No**
"Insufficient system privileges on %s."	SEVERE	**No**
"Node cannot read and write to directory: '%s'."	WARNING	**No**
"Cassandra node connection lost (%d times in the last hour)."	WARNING	**No**

Event (as the docs print it)	Severity	Emailed
"Server time of server %d is out of sync. Time difference %d milliseconds. Please enable NTP on all cluster nodes."	not shown	**No**
"Server %d shutdown initiated." / *"Server %d startup completed. (Version: %s)."*	INFO	**No**

What this means in practice. A whole node or host going down emails you. A single component failing inside a node that is still up, such as a stopped Cassandra, an overloaded Server or a full log directory, does not. Those events surface only in CMC **Events**. The node-level checks in §2 and §3 therefore have to run on a schedule rather than wait for an email.

> **Sources:** Configure Cluster event notifications (DT docs) — rows from the CLUSTER_LIFECYCLE, SERVER_LIFECYCLE and CASSANDRA tables, read 09/28/2026.

7. Memory and Time

7.1 Memory

The sizing rules come first: *"CPU and RAM must be exclusively available for Dynatrace."* *"Hosts must have at least 32 GB RAM."* With Log Monitoring, *"All Managed Cluster nodes must have at least 64 GB total RAM."* A node sharing its host with other workloads breaks the first rule before any event fires.

The Server reports heap pressure through four events, from mildest to worst: a *soft* cleanup, a *hard* cleanup, entering *emergency mode*, and ending it (*"Heap memory: Server %d ended memory emergency mode."*). None of them email you (§6). What emergency mode actually does to processing is not documented. In community practice, any hard cleanup or emergency-mode event is treated as a capacity signal — take it to MCH-06.

To compare heap across nodes, use `GET /api/v1.0/onpremise/cluster`: each node's `jvmInfo` carries its JVM settings, and the docs' example includes `maxHeap`. Memory also caps throughput. *"Dynatrace Managed environments can process a limited number of service calls per minute (depending on the node CPU amount and memory availability)"*. Past that limit, adaptive load reduction starts dropping traces (MCH-01 §7).

7.2 Time

Multi-node clusters have two clock rules. All nodes must *"Synchronize with NTP"* and *"Be in the same time zone"*. Drift is reported as *"Server time of server %d is out of sync. Time difference %d milliseconds. Please enable NTP on all cluster nodes."* That event is not emailed either. Put NTP monitoring on the hosts themselves rather than relying on the cluster to report it.

> **Sources:** Hardware requirements (DT docs), Configure Cluster event notifications (DT docs), Get cluster information about known cluster nodes (DT docs), Adaptive traffic management for Managed (DT docs).

8. Inter-Node Network

In community practice, a node that can't reach its peers looks unhealthy from every angle — Cassandra connection-lost events (§6), missing Elasticsearch shards, possibly the node dropping out of the cluster — even though every process on it is running. The docs are direct about the requirement: *"For a typical Managed deployment, all ports should be open between cluster nodes."*

Port	Used by
443	Web UI, OneAgent, REST API (routed to local 8022)
5701–5710	Hazelcast
5711, 8018	Nodekeeper
7000, 7001, 9042	Cassandra-based storage
7199	Cassandra JMX
8019	Upgrade UI
8020, 8021	Managed UI and REST API
8022	Managed UI and REST API (NGINX)
8443	OneAgent monitoring data and inter-node traffic
9200, 9300	Elasticsearch
9998	Embedded ActiveGate

Two related rules: all nodes must *"Have a network latency between nodes of 10 ms or less"*, and the node firewall itself is a Dynatrace service (`dynatrace-firewall.service` , started first in §4). If `dynatrace.sh check` reports that its rules aren't active, fix that before anything else.

> **Sources:** Cluster node ports (DT docs), Hardware requirements (DT docs), Start/stop/restart a node (DT docs).

9. When a Node Looks Unhealthy

No Managed Cluster troubleshooting page turned up in a 09/28/2026 read of the `/managed/managed-cluster/` tree, so the order below is this series' own. It checks the cheapest and most common causes first:

1. **Is the host up?** A *"Host is down."* event, or no response at all, is a host or network problem, not a Dynatrace one.
2. **Which service is down?** `dynatrace.sh status` names the unit. Look up that component's layer: Cassandra in MCH-03, Elasticsearch in MCH-04.
3. **Are the rules and ports right?** `dynatrace.sh check` , plus the inter-node port list in §8.
4. **Can it write to disk?** *"Node cannot read and write to directory"* and the log-volume event point to storage (MCH-03, MCH-04).
5. **Are clocks in sync?** Look for the out-of-sync event (§7.2).
6. **What changed?** Read CMC **Events** around the time the node went bad, including the events that were never emailed.
7. **Escalate with evidence.** Collect a diagnostic archive and open a support case.

Diagnostic archive. In the CMC, *"select More (…) > Run Cluster diagnostics"*. *"By default, 7 days of data is collected"*, packed into a `SupportArchive<ID>` ZIP with per-node folders (`autoupdater` , `config` , `log` , `managed-logs` , `nginx-error-logs`). Download it promptly: *"all diagnostic data is deleted automatically after 30 days"*.

Log locations the docs actually give: `/var/opt/dynatrace-managed/log/elasticsearch/*` and `/var/opt/dynatrace-managed/log/cassandra/*` , plus `/var/log/dynatrace/install.log` for installer operations. The docs name further logs (`nodekeeper.0.0.log` , `server.log`) without their directories. The diagnostic archive collects them for you.

> **Sources:** Diagnostic archives for Managed installations (DT docs), Multi-data center failover (DT docs), Reconfigure a node's IP address (DT docs), Configure Cluster event notifications (DT docs).

10. What the Documentation Does Not Say

Searched for in the Managed documentation on 09/28/2026 and **not found**:

Gap	Working assumption in this notebook
An official list of `operationState` values	A community-posted list of 12 exists (§2); treat anything other than `RUNNING` as unhealthy
A documented procedure for restarting **one** node in a 3+ node cluster	Follow the OS-patch guidance: one node at a time, other nodes healthy first. A community answer describes a CMC per-node Restart (§5) — a recorded conflict with the docs page
What "memory emergency mode" does to processing	The community-posted `EMERGENCY` state describes it only as *an emergency situation, e.g. low memory* (§2); treat it as a capacity alarm (MCH-06)
A Managed troubleshooting page, or the directories of `server.log` and the Nodekeeper logs (the one Server-side path found is `<datastore_dir>/log/server/audit.rest.proxy.log` , on the Mission Control data-exchange page)	Use the diagnostic archive
A dedicated "disable node" API	The documented route is the `agent` / `webUI` flags on the configuration endpoint (§5.3)

Two documentation quirks to be aware of. The `get-cluster-nodes-configuration-current-status` page's own `curl` example calls `/configuration`, while its Request URL is `/configuration/status` (use the latter). The node-script page's sample output predates the current install paths (§3).

> **Sources:** Get cluster nodes configuration current status (DT docs), Get cluster information about known cluster nodes (DT docs), Mission Control data exchange (DT docs), Configure Cluster capabilities (DT docs). **Observed 09/28/2026:** none of the five gaps is filled anywhere in the 140 pages under `/managed/managed-cluster/` and `/managed/dynatrace-api/cluster-api/` (crawled and searched for `operationState` values, restart procedures, `emergency`, log paths and troubleshooting pages).

11. Recommended Approach

1. **Poll the API from outside the cluster** every few minutes. Alert on any node not `RUNNING`, on a node count below your cluster size, and on mismatched `buildVersion`.
2. **Run `dynatrace.sh status` and `check` on every node on a schedule**, and alert on anything other than *All services are OK* / *All processes are OK* / *All rules are active*. This is how you catch the component failures the cluster never emails about.
3. **Monitor NTP and inter-node reachability on the hosts themselves.** The cluster reports clock drift only through a non-emailed event.
4. **Document the start and stop procedure for *your* cluster size** before you need it. At 3+ nodes, that means the gated cluster procedure, not `dynatrace.sh restart`.
5. **Enforce one node at a time** for every planned operation, and confirm the others are healthy before starting.
6. **Read CMC Events weekly**, and immediately after any node incident.

12. Summary and Next Steps

Node health is two questions. Is every node present and `RUNNING`? Is every service on it up, listening, and behind active firewall rules? The API and CMC answer the first; `dynatrace.sh status` and `check` answer the second. Services start in a gated order, and in clusters of three or more nodes they are stopped with the cluster procedure. Planned work always goes one node at a time. Most process-level failures never email you, so these checks have to run on a schedule.

Next in the series:

Notebook	Layer
MCH-03: Cassandra Metrics Store Health	2 — Storage

Notebook	Layer
MCH-04: Elasticsearch Store Health	2 — Storage
MCH-06: Capacity and Scaling	3 — Capacity (where memory-pressure events lead)
MCH-05: Cluster ActiveGates and Mission Control Connectivity	4 — Connectivity

---

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.